

מדריך לעבודה עם ה-Canvas (משטח הצیור)

כללי:

משטח הציור (Canvas) בפרויקט נועד לאפשר למשתמש לצייר את תשובתו באמצעות מגע ולשלוח את הציור לשרת לצורך ניתוח (תכונת Draw-By-Touch canvas).

מימוש ה-Canvas

משטח הציור מיושם באמצעות האובייקט lv_canvas של ספריית LVGL:

1. יצירת אובייקט: הפונקציה initialize_and_place_canvas () יוצרת את הרכיב.
2. הקצאת באפר: LVGL Canvas דורש מערך פיקסלים (Buffer) רציף בזיכרון הראשי (SRAM/DRAM) כדי לאחסן את התמונה כולה. הבאפר מוקצה באופן דינמי באמצעות malloc על מנת לשמור על נתוני הציור:

```
size_t buf_size = LV_CANVAS_BUF_SIZE_TRUE_COLOR(width, height);  
  
// Allocate the buffer dynamically on the heap  
lv_color_t* canvas_buffer = (lv_color_t*) malloc(buf_size);
```

פורמט הצבע שנבחר הוא LV_IMG_CF_TRUE_COLOR (16-565 RGB bit), מה שאומר שכל פיקסל דורש 2 בתים של זיכרון.

3. אירוע ציור: פונקציית ה-callback (canvas_draw_event_cb) מטפלת באירועי מגע:

- זיהוי לחיצה (LV_EVENT_PRESSING): בזמן לחיצה מתמשכת, הקוד מחשב את נקודת המגע הנוכחית ביחס למשטח הציור.
- ציור קווים: הפונקציה lv_canvas_draw_line () משמשת לשרטט קו ישר מהנקודה הקודמת (last_x, last_y) לנקודה הנוכחית, תוך שימוש במתאר הקו (line_dsc) המוגדר (כחול, עובי 4).
- סיום ציור (LV_EVENT_RELEASED): עם שחרור המגע, הקואורדינטות האחרונות מאופסות (last_x = -1, last_y = -1) כדי להבטיח שקו חדש יתחיל בלחיצה הבאה.

4. זיכרון: מנגנון ניהול זיכרון מותאם אישית נוסף באמצעות LV_EVENT_DELETE כדי לוודא שבאפר הפיקסלים שהוקצה באמצעות malloc משוחרר כאשר אובייקט ה-Canvas נמחק, ובכך נמנעות דליפות זיכרון.

האתגר: מגבלת הזיכרון (DRAM)

ה-ESP32, במיוחד בשילוב עם ספריית LVGL, מתמודד עם מגבלת זיכרון (DRAM (Dynamic RAM), המשמש גם להקצאת באפרים גדולים.

נפח הקנבס זה הוא משמעותי מאוד ועלול לגרום לכשל בהקצאה (canvas_buffer == nullptr), במיוחד כאשר הזיכרון נדרש גם לבאפר התצוגה של disp_draw_buf ולפעולות רשת נוספות.

- הפתרון בפרויקט - צמצום גודל ה-Canvas: כדי להבטיח יציבות ולאפשר הקצאת זיכרון מוצלחת, צומצם גודל ה-Canvas לגודל קטן ומספק עבור ציור אות בודדת.

C

```
initialize_and_place_canvas(tab3, 120, 120);
```

הצמצום (מ-150KB ל-28KB) הפחית באופן דרסטי את הדרישה לזיכרון, איפשר הקצאה אמינה, ופתר את בעיית ה-DRAM.

שליחת הציור לשרת

כפתור הבדיקה (check_btn) מפעיל את הפונקציה handle_check_btn(), אשר אחראית על שליחת הציור לשרת (תכונת Send canvas to a server):

1. קבלת נתונים: הקוד מאחזר את המבנה המכיל את נתוני הפיקסלים של ה-Canvas:

```
const lv_img_dsc_t* img_dsc = lv_canvas_get_img(global_canvas);
if (!img_dsc || !img_dsc->data) {
    Serial.println("Error: Canvas buffer data not found.");
    return;
}
```

פרוטוקול TCP: ה-ESP32 מתחבר מחדש לשרת ה-TCP באמצעות WiFiClient (בכתובת (SERVER_IP:SERVER_PORT).

2. שליחת Header: נשלחת הודעת Header פשוטה המכילה את הרוחב, הגובה והגודל הכולל של הבאפר. זה מאפשר לשרת ה-Python בצד השני לדעת בדיוק כמה בתים לצפות לקרוא וכיצד לפרש את הנתונים כמשטח תמונה.
3. שליחת נתונים גולמיים: הנתונים הבינאריים של באפר הפיקסלים (buffer) נשלחים באופן גולמי (Raw Data) לשרת באמצעות client.write(). השרת יכול לאחר מכן לעבד את הנתונים הללו, לבצע זיהוי תבניות או אותיות, ולהחזיר תגובה.

4. אישור סגירה: לאחר השליחה, הקוד ממתין לקבלת אישור (ACK) מהשרת וסוגר את החיבור
(client.stop()) כדי לשחרר משאבי רשת.